

Master Notes — Linked Lists & Trees

Part 1b: Trees — BST, Balanced Trees, Heaps, Tries, Segment/Fenwick Trees, Advanced Algorithms (C++, FAANG-condensed)

Contents

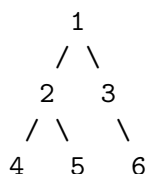
1	Binary Trees — Fundamentals & Traversals	2
2	Binary Search Trees (BST)	8
3	Self-Balancing BSTs (AVL, Red-Black, Treap, B-Trees)	12
4	Heaps / Priority Queues	15
5	Tries (Prefix Trees)	18
6	Segment Trees	22
7	Fenwick Trees (Binary Indexed Trees)	26
8	Advanced Tree Algorithms	29

1 Binary Trees — Fundamentals & Traversals

One-line definition

A hierarchical structure where each node has at most two children (**left**, **right**); a connected, acyclic graph with N nodes and $N - 1$ edges rooted at one node.

Intuition



Recursion is the natural language of trees: almost every algorithm is "solve for left subtree, solve for right subtree, combine at the root."

Graph Intuition

- A tree is a special-case DAG: exactly one path between any two nodes, no cycles.
- Recursive definition: a tree is either empty, or a root with two disjoint subtrees that are themselves trees — this is why induction/recursion proofs on trees "just work."
- Height h of a balanced tree is $O(\log n)$; height of a degenerate (skewed) tree is $O(n)$ — this single fact drives the motivation for every self-balancing structure in Section 3.

Memory Layout

- Heap-allocated nodes, non-contiguous $\Rightarrow$ cache misses on every pointer hop (same caveat as linked lists).
- Recursive traversal depth = tree height $\Rightarrow$ stack overflow risk on skewed trees with $n > 10^5$; prefer iterative/Morris traversal when height is unbounded.

Operations

0. Node Definition

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};
```

1. DFS Traversals (recursive & iterative)

Complexity: $O(n)$ time; $O(h)$ space (recursion stack or explicit stack).

```

void preorder(TreeNode* r, vector<int>& out) {
    if (!r) return;
    out.push_back(r->val); preorder(r->left, out); preorder(r->right, out)
    ;
}
vector<int> preorderIter(TreeNode* root) {
    vector<int> out; if (!root) return out;
    stack<TreeNode*> st; st.push(root);
    while (!st.empty()) {
        TreeNode* n = st.top(); st.pop(); out.push_back(n->val);
        if (n->right) st.push(n->right);
        if (n->left) st.push(n->left);
    }
    return out;
}
vector<int> inorderIter(TreeNode* root) {
    // left, root, right
    vector<int> out; stack<TreeNode*> st; TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }
        cur = st.top(); st.pop(); out.push_back(cur->val); cur = cur->
            right;
    }
    return out;
}

```

Postorder iterative: push to a stack, prepend to output (reverse of a "root, right, left" traversal)
— simplest to implement without a "last visited" pointer.

2. BFS / Level-Order

Complexity: $O(n)$ time, $O(n)$ space (widest level). **Zigzag variant:** same code, reverse alternate levels.

```

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res; if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size(); vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* n = q.front(); q.pop(); level.push_back(n->val);
            if (n->left) q.push(n->left);
            if (n->right) q.push(n->right);
        }
        res.push_back(level);
    }
    return res;
}

```

3. Morris Inorder Traversal ($O(1)$ space)

Idea: temporarily thread a null right pointer of the inorder predecessor to the current node, enabling traversal without a stack; undo the thread on the way back. $O(n)$ time, $O(1)$ space — the go-to answer when asked "can you avoid the stack entirely?"

```

void morrisInorder(TreeNode* root) {
    TreeNode* cur = root;
    while (cur) {
        if (!cur->left) { visit(cur); cur = cur->right; }
        else {
            TreeNode* pred = cur->left;
            while (pred->right && pred->right != cur) pred = pred->right;
            if (!pred->right) { pred->right = cur; cur = cur->left; }
            else { pred->right = nullptr; visit(cur); cur = cur->right; }
        }
    }
}

```

4. Height, Diameter, Balanced Check (single-pass patterns)

Diameter = longest path between any two nodes, may not pass through root; compute via a single post-order pass that returns height while updating a running best. $O(n)$ time, $O(h)$ space.

```

int diameterHelper(TreeNode* r, int& best) {
    if (!r) return 0;
    int lh = diameterHelper(r->left, best), rh = diameterHelper(r->right,
        best);
    best = max(best, lh + rh);
    return 1 + max(lh, rh);
}

int checkBalanced(TreeNode* r) {                // returns -1 if unbalanced
    anywhere, else height
    if (!r) return 0;
    int lh = checkBalanced(r->left); if (lh == -1) return -1;
    int rh = checkBalanced(r->right); if (rh == -1) return -1;
    if (abs(lh - rh) > 1) return -1;
    return 1 + max(lh, rh);
}

```

Trick: "compute X while also computing height" is the dominant single-pass pattern for tree problems — avoids the naive $O(n^2)$ / $O(n \log n)$ of recomputing height per node.

5. Lowest Common Ancestor (general binary tree)

Idea: post-order; a node is the LCA if p and q are found in different subtrees (or the node itself is p or q). $O(n)$ time, $O(h)$ space, no parent pointers needed.

```

TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* l = lca(root->left, p, q);
    TreeNode* r = lca(root->right, p, q);
    if (l && r) return root;
    return l ? l : r;
}

```

6. Max Path Sum, Same/Symmetric Tree, Invert

```

int maxPathSumHelper(TreeNode* r, int& best) {
    if (!r) return 0;
    int l = max(0, maxPathSumHelper(r->left, best));
    int rr = max(0, maxPathSumHelper(r->right, best));
    best = max(best, r->val + l + rr);
    return r->val + max(l, rr);
}

TreeNode* invert(TreeNode* r) {
    if (!r) return nullptr;
    swap(r->left, r->right);
    invert(r->left); invert(r->right);
    return r;
}

```

Same tree: compare values + recurse both sides, short-circuit on structural mismatch. **Symmetric:** same idea comparing a tree against its own mirror.

7. Serialize / Deserialize

Idea: preorder with an explicit # sentinel for null children makes the tree uniquely reconstructible without needing a second traversal order. $O(n)$ time/space.

```

string serialize(TreeNode* root) {
    if (!root) return "#";
    return to_string(root->val) + "," + serialize(root->left) + "," +
        serialize(root->right);
}

TreeNode* deserializeHelper(queue<string>& q) {
    string token = q.front(); q.pop();
    if (token == "#") return nullptr;
    TreeNode* node = new TreeNode(stoi(token));
    node->left = deserializeHelper(q);
    node->right = deserializeHelper(q);
    return node;
}

```

8. Construct Tree from Preorder + Inorder

Idea: preorder[0] is the root; find it in inorder (hashmap for $O(1)$ lookup) to split into left/right subtree ranges; recurse. $O(n)$ time with hashmap (vs. $O(n^2)$ with linear search), $O(n)$ space.

Patterns

Interviewer says	Think
"can you do it without recursion / $O(1)$ space"	iterative with stack, or Morris traversal
"longest path / diameter"	single post-order pass returning height
"reconstruct tree from traversals"	hashmap index + divide on root position
"LCA, no parent pointers"	post-order recursion, "found in both subtrees"
"compute X for every node efficiently"	single-pass post-order combining X with height

Complexity Table

Operation	Time	Space	Notes
DFS (any order)	$O(n)$	$O(h)$	$O(n)$ worst case if skewed
BFS / level order	$O(n)$	$O(n)$	widest level
Morris inorder	$O(n)$	$O(1)$	temporarily mutates tree
Diameter / balanced check	$O(n)$	$O(h)$	single pass
Serialize/Deserialize	$O(n)$	$O(n)$	

Interview Follow-ups — Answered

- **”Can you do this traversal iteratively?”** Yes for all three DFS orders using an explicit stack; preorder/inorder are direct, postorder needs either two stacks or a ”last visited” pointer trick.
- **”Can you do it in $O(1)$ space?”** Morris traversal for inorder/preorder; for problems like diameter or balanced-check, $O(h)$ recursion stack is unavoidable without an explicit iterative post-order simulation (which still costs $O(h)$ space, just off the call stack).
- **”What if the tree is heavily skewed?”** Height becomes $O(n)$, so recursion risks stack overflow and BFS becomes memory-heavy in the wrong direction — mention converting to iterative traversal.

Common Bugs

- Forgetting the base case `!root` in recursive functions $\Rightarrow$ segfault.
- Confusing preorder/inorder/postorder push order in the iterative versions.
- Not restoring `pred->right = nullptr` in Morris traversal $\Rightarrow$ corrupts tree / infinite loop.
- Off-by-one when splitting inorder ranges during tree reconstruction.

Checklist

1. Null root handled first?
2. Recursive or iterative required by the constraints?
3. Can this be solved in a single pass (combine two computations)?
4. Does the answer need the path itself, or just a value (sum/height/count)?

Practice Problems

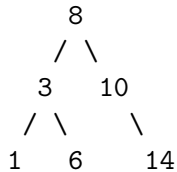
Difficulty	Problem	Source
Easy	Maximum Depth of Binary Tree	LeetCode 104
Easy	Invert Binary Tree	LeetCode 226
Easy	Symmetric Tree	LeetCode 101
Medium	Binary Tree Level Order Traversal	LeetCode 102
Medium	Diameter of Binary Tree	LeetCode 543
Medium	Construct Binary Tree from Preorder and Inorder	LeetCode 105
Medium	Lowest Common Ancestor of a Binary Tree	LeetCode 236
Hard	Binary Tree Maximum Path Sum	LeetCode 124
Hard	Serialize and Deserialize Binary Tree	LeetCode 297

2 Binary Search Trees (BST)

One-line definition

A binary tree with the ordering invariant: for every node, all left-subtree values $<$ node $<$ all right-subtree values — this invariant makes inorder traversal produce sorted output.

Intuition



The ordering invariant turns "search" into a repeated binary decision (go left or right), giving $O(\log n)$ operations on a balanced BST — same asymptotics as binary search on a sorted array, but with $O(\log n)$ insert/delete that arrays can't match.

Graph Intuition

A BST is a binary tree plus a total order constraint; **inorder traversal always yields sorted order** regardless of shape — this is the single most-used property in BST problems (validate, kth smallest, closest value, range sum).

Operations

1. Insert / Search / Delete

Complexity: $O(h)$, i.e. $O(\log n)$ balanced, $O(n)$ worst case (skewed — degenerates to a linked list).

Delete's 3 cases: leaf (remove directly), one child (splice child up), two children (replace with inorder successor/predecessor, then delete that node recursively).


```

TreeNode* insert(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}

TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) root->left = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else {
        if (!root->left) return root->right;
        if (!root->right) return root->left;
        TreeNode* succ = root->right;
        while (succ->left) succ = succ->left;
        root->val = succ->val;
        root->right = deleteNode(root->right, succ->val);
    }
    return root;
}

```

2. Validate BST

Trap: comparing only against immediate parent is wrong — must carry a valid (lo, hi) range down the whole recursion (use long long bounds to dodge INT_MIN/INT_MAX edge cases).

```

bool valid(TreeNode* r, long long lo, long long hi) {
    if (!r) return true;
    if (r->val <= lo || r->val >= hi) return false;
    return valid(r->left, lo, r->val) && valid(r->right, r->val, hi);
}

```

3. Kth Smallest, Floor/Ceil, Successor/Predecessor

Kth smallest: inorder traversal, stop at the k -th visit — $O(h + k)$ time, or Morris for $O(1)$ space.

Floor/Ceil: walk down comparing, keep the best candidate seen so far — $O(h)$.

Successor/Predecessor: if node has a right child, successor = leftmost of right subtree; else it's the lowest ancestor for which the node is in the left subtree.

4. LCA in a BST

Idea: exploit ordering — no need to search both subtrees like the general binary-tree LCA. $O(h)$ time, $O(1)$ space (iterative).

```

TreeNode* lcaBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val) root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;
    }
    return nullptr;
}

```

5. Sorted Array/List → Balanced BST

Pick the middle element as root, recurse on both halves — guarantees a height-balanced BST in $O(n)$ time.

6. BST Iterator (controlled inorder, $O(1)$ amortized `next()`)

Idea: maintain an explicit stack primed with the leftmost path; `next()` pops, then pushes the leftmost path of the popped node's right subtree. $O(h)$ space, each node pushed/popped once overall $\Rightarrow$ amortized $O(1)$ per `next()` call.

Patterns

Interviewer says	Think
"kth smallest/largest"	inorder traversal with early stop
"validate this is a BST"	range-bound recursion, not just parent comparison
"closest value / floor / ceil"	walk down, track best-so-far
"LCA in a BST"	ordering property, no need to search both sides
"convert sorted data to balanced tree"	pick middle as root, recurse

Complexity Table

Operation	Balanced	Skewed (worst case)
Search / Insert / Delete	$O(\log n)$	$O(n)$
Kth smallest (inorder)	$O(h + k)$	$O(n)$
LCA	$O(h)$	$O(n)$

Interview Follow-ups — Answered

- **"What if the BST becomes skewed?"** All operations degrade to $O(n)$ — motivates self-balancing trees (Section 3); mention this proactively when asked about worst case.
- **"Can duplicates exist?"** Convention-dependent — clarify with interviewer; typical convention places duplicates in the right subtree (or disallows them entirely).
- **"How is this different from a general binary tree for search?"** The ordering invariant turns $O(n)$ search into $O(\log n)$ (balanced) — the entire value proposition of a BST.

Common Bugs

- Validating BST by comparing only to the immediate parent instead of the full ancestor range.
- Off-by-one in strict vs. non-strict inequality ($\leq$ vs $<$) when duplicates are allowed.
- Forgetting to reattach the subtree after deleting a two-children node (must delete the successor from its *original* position, not just overwrite the value).

Checklist

1. Balanced assumption, or must I handle skew?
2. Recursive range-bound check for validation, not parent-only comparison?
3. Can I exploit inorder = sorted instead of a generic tree algorithm?

Practice Problems

Difficulty	Problem	Source
Easy	Search in a Binary Search Tree	LeetCode 700
Easy	Convert Sorted Array to BST	LeetCode 108
Medium	Validate Binary Search Tree	LeetCode 98
Medium	Kth Smallest Element in a BST	LeetCode 230
Medium	Delete Node in a BST	LeetCode 450
Medium	Lowest Common Ancestor of a BST	LeetCode 235
Medium	BST Iterator	LeetCode 173
Hard	Recover Binary Search Tree	LeetCode 99

3 Self-Balancing BSTs (AVL, Red-Black, Treap, B-Trees)

One-line definition

BST variants that enforce an extra structural invariant on every insert/delete so height stays $O(\log n)$, guaranteeing all BST operations remain logarithmic.

Intuition

A plain BST's worst case is a linked list ($O(n)$). Self-balancing trees pay a small constant-factor cost on every insert/delete (rotations, recoloring, or randomization) to *guarantee* $O(\log n)$ height — this is the standard answer to "what if the BST becomes skewed?"

Operations

1. AVL Tree

Invariant: for every node, $|height(left) - height(right)| \leq 1$ (the "balance factor").

Idea: after insert/delete, walk back up and fix the first unbalanced node with a rotation. **4**

rotation cases: LL (right rotate), RR (left rotate), LR (left rotate child, then right rotate), RL (mirror of LR). $O(\log n)$ insert/delete/search, guaranteed.

```
struct AVLNode { int val, height; AVLNode *left, *right; };
int height(AVLNode* n) { return n ? n->height : 0; }
int balanceFactor(AVLNode* n) { return n ? height(n->left) - height(n->right) : 0; }

AVLNode* rotateRight(AVLNode* y) {
    AVLNode* x = y->left; AVLNode* T2 = x->right;
    x->right = y; y->left = T2;
    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));
    return x;                                // new subtree root
}
// rotateLeft is the mirror image of rotateRight
```

When interviewers ask for AVL: usually want the balance-factor concept and rotation cases explained verbally with a diagram, not a full working implementation — know the 4 cases cold.

2. Red-Black Tree

Properties (all must hold):

1. Every node is red or black; root is black.
2. Red node's children are both black (no two consecutive reds on any path).
3. Every path from a node to its descendant **null** leaves has the same number of black nodes (the "black-height").

Why $O(\log n)$ height: since no path can have two reds in a row, the longest possible path is at most twice as long as the shortest one — so the tree can never get too skewed, and height stays $O(\log n)$.

Used in: `std::map`/`std::set` (C++ STL), Java `TreeMap`, Linux CFS scheduler. Rebalancing uses rotations + recoloring, fewer rotations per insert than AVL (better for write-heavy workloads);

AVL is more tightly balanced (better for read-heavy workloads). **Interview expectation:** explain properties and the AVL-vs-RB tradeoff — full implementation is rarely asked live.

3. Treap (Tree + Heap)

Idea: each node gets a random priority; the tree maintains BST order on keys *and* max-heap order on priorities simultaneously. Because priorities are random, expected height is $O(\log n)$ with high probability — balance “for free” without explicit rotation-case logic (rotations still happen, but the *decision* of when is driven purely by priority comparison, which is much simpler code than AVL/RB). Useful in competitive programming for order-statistics trees and mergeable BSTs.

4. B-Trees / B+ Trees (overview)

Idea: a multi-way search tree where each node holds many keys (branching factor t), keeping height very small ($O(\log_t n)$) — designed so each node maps to one disk page/block, minimizing disk I/O. **B+ Tree:** all actual data lives in leaf nodes, which are additionally linked in a list for fast sequential range scans; internal nodes hold only routing keys. This is exactly how MySQL InnoDB and most filesystem/database indexes are implemented.

Patterns

Interviewer says	Think
“guarantee $O(\log n)$ even with adversarial input”	AVL / Red-Black
“why does <code>std::map</code> guarantee $O(\log n)$?”	Red-Black tree internally
“database index design”	B+ Tree, disk-block-aligned
“randomized balance without rotation-case logic”	Treap

Complexity Table

Structure	Height	Insert/Delete	Notes
AVL	$O(\log n)$, tightly balanced	$O(\log n)$, up to $O(\log n)$ rotations	best for read-heavy
Red-Black	$O(\log n)$, $\leq 2 \log_2(n+1)$	$O(\log n)$, $O(1)$ amortized rotations	best for write-heavy
Treap	$O(\log n)$ expected	$O(\log n)$ expected	simplest to code
B/B+ Tree	$O(\log_t n)$	$O(\log_t n)$	disk/database optimized

Interview Follow-ups — Answered

- **“Why does `std::map` give $O(\log n)$ guarantees?”** It’s implemented as a Red-Black tree internally — balanced height is enforced on every insert/erase.
- **“AVL vs Red-Black — which would you pick?”** AVL for lookup-heavy workloads (tighter balance $\Rightarrow$ shorter average search path); Red-Black for insert/delete-heavy workloads (fewer rotations per update).
- **“Why do databases use B+ trees instead of a balanced binary tree?”** Branching factor t much larger than 2 means far fewer disk-page reads for the same n ($\log_t n \ll \log_2 n$), and each node is sized to match a disk block — this is an I/O-bound optimization, not a CPU/comparison one.

Common Bugs

- Forgetting to update cached height after a rotation (AVL) $\Rightarrow$ wrong future balance-factor decisions.
- Rotating in the wrong direction for LR/RL double-rotation cases.
- Assuming Treap gives worst-case (not just expected) $O(\log n)$ — it's a probabilistic guarantee.

Checklist

1. Do I need to *implement* balancing, or just explain the invariant and tradeoffs?
2. Read-heavy (AVL) vs write-heavy (Red-Black) — which fits the stated workload?
3. Disk/IO-bound context $\Rightarrow$ B/B+ tree, not an in-memory balanced BST.

Practice Problems

Difficulty	Problem	Source
Medium	Balance a Binary Search Tree	LeetCode 1382
Medium	Implement an AVL insert (explain + code rotations)	Interview staple
Hard	Design a data structure with order statistics	Treap / augmented BST
Conceptual	Explain Red-Black invariants & why <code>std::map</code> uses it	System-design/CS-fundamentals round

4 Heaps / Priority Queues

One-line definition

A complete binary tree (stored implicitly in an array) satisfying the heap property: every parent is $\leq$ (min-heap) or $\geq$ (max-heap) its children — giving $O(1)$ peek and $O(\log n)$ insert/extract of the min/max element.

Intuition

Array: [1, 3, 2, 7, 5, 8]	Tree view:
index: 0 1 2 3 4 5	1
	/ \
parent(i) = (i-1)/2	3 2
left(i) = 2i+1	/ \ /
right(i) = 2i+2	7 5 8

”Complete” (filled left-to-right, no gaps) is what lets the tree live in a flat array with no pointers — index arithmetic replaces left/right pointers entirely.

Operations

1. Sift Up / Sift Down (Heapify) & Build Heap

Complexity: sift up/down $O(\log n)$; **build-heap from an unsorted array is $O(n)$, not $O(n \log n)$** — a classic ”explain why” interview question. Most nodes sit near the bottom of the tree, where sift-down barely has to move anything; only the few nodes near the root do real work. Adding that up across all levels comes out to $O(n)$ total, not $O(n \log n)$.

```
void siftDown(vector<int>& h, int i, int n) {           // min-heap
    while (true) {
        int l = 2*i+1, r = 2*i+2, smallest = i;
        if (l < n && h[l] < h[smallest]) smallest = l;
        if (r < n && h[r] < h[smallest]) smallest = r;
        if (smallest == i) break;
        swap(h[i], h[smallest]); i = smallest;
    }
}
void buildHeap(vector<int>& h) {
    for (int i = h.size()/2 - 1; i >= 0; i--) siftDown(h, i, h.size());
}
```

Heap sort: build a max-heap, repeatedly swap root with the last element and sift-down the reduced heap — $O(n \log n)$ time, $O(1)$ extra space, in-place but *not* stable.

2. std::priority_queue Usage

```

priority_queue<int> maxHeap; // max-heap by default
priority_queue<int, vector<int>, greater<int>> minHeap; // min-heap
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; // custom compare, e.g. (dist, node)

```

3. Kth Largest Element

Idea: maintain a min-heap of size k ; if a new element beats the heap's min, pop-and-push. $O(n \log k)$ time, $O(k)$ space — better than sorting ($O(n \log n)$) when $k \ll n$.

4. Top-K Frequent Elements

Idea: count frequencies (hashmap), then either a size- k min-heap on frequency ($O(n \log k)$) or bucket sort by frequency ($O(n)$, since frequency is bounded by n).

5. Merge K Sorted Lists

Idea: min-heap holding the current head of each list; pop the smallest, push its successor. $O(N \log k)$ time where N = total nodes, k = number of lists; $O(k)$ heap space.

6. Median of a Data Stream (two heaps)

Idea: a max-heap for the lower half, a min-heap for the upper half, kept balanced in size (differ by at most 1); median is either heap's top, or the average of both tops. $O(\log n)$ insert, $O(1)$ query median — the canonical "two heaps" pattern.

Patterns

Interviewer says	Think
"kth largest/smallest, streaming"	size- k heap
"merge k sorted structures"	min-heap of k current candidates
"running median"	two heaps (max-heap low, min-heap high)
"top k frequent"	heap or bucket sort by frequency
"schedule by priority / shortest job next"	priority queue

Complexity Table

Operation	Time	Space
Peek min/max	$O(1)$	$O(1)$
Insert	$O(\log n)$	$O(1)$
Extract min/max	$O(\log n)$	$O(1)$
Build heap from array	$O(n)$	$O(1)$ extra
Heap sort	$O(n \log n)$	$O(1)$ extra

Interview Follow-ups — Answered

- **"Why is build-heap $O(n)$ and not $O(n \log n)$?"** A node's sift-down cost depends on its height, and most nodes are near the bottom (height close to 0), so most nodes cost almost nothing — the total work across the whole tree still adds up to only $O(n)$.
- **"Is heap sort stable?"** No — swaps during sift-down can reorder equal elements; use merge sort if stability is required.
- **"Array-based heap vs pointer-based tree — why array?"** Array gives $O(1)$ parent/child index arithmetic, perfect cache locality (contiguous memory), and no pointer overhead — strictly better for a complete binary tree than a pointer-based node structure.

Common Bugs

- Confusing `priority_queue<int>` (max-heap) with wanting a min-heap — forgetting the `greater<int>` comparator.
- Off-by-one in $2*i+1/2*i+2$ index math.
- Sifting down instead of up (or vice versa) after insert/extract — insert sifts up, extract sifts down.

Checklist

1. Min-heap or max-heap needed?
2. Fixed-size- k heap applicable to bound memory/time?
3. Would bucket sort beat a heap here (bounded value range)?

Practice Problems

Difficulty	Problem	Source
Easy	Kth Largest Element in a Stream	LeetCode 703
Medium	Kth Largest Element in an Array	LeetCode 215
Medium	Top K Frequent Elements	LeetCode 347
Medium	Task Scheduler	LeetCode 621
Hard	Merge k Sorted Lists	LeetCode 23
Hard	Find Median from Data Stream	LeetCode 295
Hard	Sliding Window Maximum (heap variant)	LeetCode 239

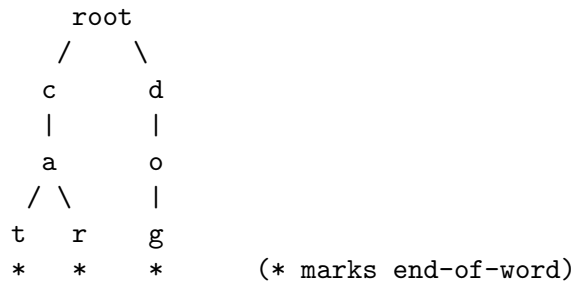
5 Tries (Prefix Trees)

One-line definition

A tree where each edge represents one character, and each root-to-node path represents a string prefix — built specifically to make prefix operations $O(L)$ instead of $O(n \cdot L)$.

Intuition

Words: "cat", "car", "dog"



Shared prefixes ("ca") are stored once — searching for any word or prefix costs only $O(L)$, the length of the query, completely independent of how many other words are stored.

Operations

1. Insert / Search / StartsWith

Complexity: $O(L)$ per operation, L = word length. **Space:** up to $O(26 \cdot N \cdot L)$ worst case (no shared prefixes) with a fixed-size array; a `hashmap<char, Node*>` trades a little speed for much better space on sparse alphabets.

```

struct TrieNode {
    TrieNode* children[26] = {};
    bool isEnd = false;
};
class Trie {
    TrieNode* root = new TrieNode();
public:
    void insert(const string& w) {
        TrieNode* cur = root;
        for (char c : w) {
            int i = c - 'a';
            if (!cur->children[i]) cur->children[i] = new TrieNode();
            cur = cur->children[i];
        }
        cur->isEnd = true;
    }
    TrieNode* traverse(const string& w) {
        TrieNode* cur = root;
        for (char c : w) {
            int i = c - 'a';
            if (!cur->children[i]) return nullptr;
            cur = cur->children[i];
        }
        return cur;
    }
    bool search(const string& w) { TrieNode* n = traverse(w); return n && n->isEnd; }
    bool startsWith(const string& p) { return traverse(p) != nullptr; }
};

```

2. Word Search II (Trie + Backtracking on a Grid)

Idea: build a trie of the dictionary, DFS/backtrack from every grid cell while walking the trie in lockstep — prunes any path whose prefix isn't in the dictionary, turning a brute-force $O(\text{words} \times \text{grid})$ search into a single combined traversal.

3. Longest Common Prefix, Autocomplete, Replace Words

- **Longest common prefix of a set of strings:** insert all, walk down while each node has exactly one child and isn't a word-end.
- **Autocomplete:** `startsWith` to find the prefix node, then DFS that subtree to collect all completions.
- **Replace Words (dictionary root replacement):** insert all roots, for each word in the sentence walk the trie until hitting `isEnd` (shortest matching root) or a dead end.

Advanced: Compressed Trie / Radix Tree

Idea: collapse chains of single-child nodes into one edge labeled with a substring instead of one character — turns $O(L)$ nodes per word into far fewer, saving huge amounts of memory on sparse key sets. Used in IP routing tables (longest-prefix match) and some in-memory key-value stores.

Patterns

Interviewer says	Think
"prefix search / autocomplete"	Trie
"search a dictionary of words on a grid"	Trie + backtracking, prune with <code>startsWith</code>
"replace words with their dictionary root"	Trie, walk until <code>isEnd</code>
"IP routing / longest prefix match"	compressed trie (radix tree)

Complexity Table

Operation	Time	Space
Insert	$O(L)$	$O(L)$ new nodes worst case
Search / <code>StartsWith</code>	$O(L)$	$O(1)$
Total for N words, length L	–	$O(26 \cdot N \cdot L)$ worst case

Interview Follow-ups — Answered

- **"Why Trie over a hashset of strings for prefix queries?"** A hashset only answers exact membership in $O(L)$; it cannot answer "does any word start with this prefix" without scanning every entry. Trie answers both in $O(L)$.
- **"How do you reduce memory for a sparse alphabet (e.g. Unicode)?"** Use `unordered_map<char, Node>` instead of a fixed 26/256-size array per node, or use a compressed trie to merge single-child chains.
- **"Can you delete a word?"** Walk to the end node, unset `isEnd`; then walk back up deleting nodes that have no children and aren't the end of another word.

Common Bugs

- Confusing "prefix exists" (`startsWith`) with "word exists" (`isEnd`) — a node being reachable doesn't mean a full word ends there.
- Not initializing the fixed-size children array to `nullptr` (undefined behavior on lookup).
- Memory leaks from never freeing trie nodes in long-running services (rarely penalized in interviews, but worth mentioning).

Checklist

1. Array (fixed alphabet) or hashmap children (sparse/Unicode)?
2. Do I need `isEnd` per node, or just prefix existence?
3. Would a compressed trie meaningfully save memory here?

Practice Problems

Difficulty	Problem	Source
Medium	Implement Trie (Prefix Tree)	LeetCode 208
Medium	Design Add and Search Words Data Structure	LeetCode 211
Medium	Replace Words	LeetCode 648
Medium	Longest Word in Dictionary	LeetCode 720
Hard	Word Search II	LeetCode 212
Hard	Design Search Autocomplete System	LeetCode 642

6 Segment Trees

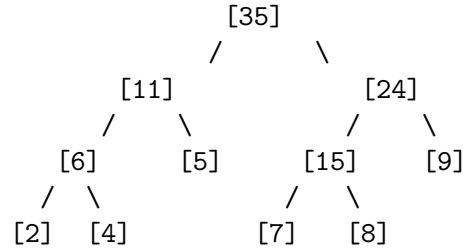
One-line definition

A binary tree over an array where each node stores an aggregate (sum/min/max/gcd) of a contiguous range, enabling both range queries and point/range updates in $O(\log n)$.

Intuition

Array: [2, 4, 5, 7, 8, 9]

Tree (sum):



Each internal node "summarizes" its children's range, so any query range can be answered by combining $O(\log n)$ pre-computed node summaries instead of scanning every element.

Operations

1. Build, Query, Point Update

Complexity: build $O(n)$, query/update $O(\log n)$. **Space:** array of size $4n$ is a safe upper bound for a 1-indexed array-based segment tree.

```

vector<int> tree;
void build(vector<int>& a, int node, int l, int r) {
    if (l == r) { tree[node] = a[l]; return; }
    int mid = (l + r) / 2;
    build(a, 2*node, l, mid);
    build(a, 2*node+1, mid+1, r);
    tree[node] = tree[2*node] + tree[2*node+1];
}
int query(int node, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0; // no overlap
    if (ql <= l && r <= qr) return tree[node]; // full overlap
    int mid = (l + r) / 2;
    return query(2*node, l, mid, ql, qr) + query(2*node+1, mid+1, r, ql, qr);
}
void update(int node, int l, int r, int idx, int val) {
    if (l == r) { tree[node] = val; return; }
    int mid = (l + r) / 2;
    if (idx <= mid) update(2*node, l, mid, idx, val);
    else update(2*node+1, mid+1, r, idx, val);
    tree[node] = tree[2*node] + tree[2*node+1];
}

```

2. Lazy Propagation (range updates)

Idea: when a range update fully covers a node's range, don't recurse into children immediately — stash the pending update in a `lazy[]` array and push it down only when that subtree is actually visited later. Turns range-update from $O(n)$ (updating every element) into $O(\log n)$.

```

vector<int> lazy;
void pushDown(int node) {
    if (!lazy[node]) return;
    for (int child : {2*node, 2*node+1}) {
        tree[child] += lazy[node];
        lazy[child] += lazy[node];
    }
    lazy[node] = 0;
}
void rangeUpdate(int node, int l, int r, int ql, int qr, int val) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) { tree[node] += val; lazy[node] += val; return; }
    pushDown(node);
    int mid = (l + r) / 2;
    rangeUpdate(2*node, l, mid, ql, qr, val);
    rangeUpdate(2*node+1, mid+1, r, ql, qr, val);
    tree[node] = tree[2*node] + tree[2*node+1];
}

```

Patterns

Interviewer says	Think
"range sum/min/max query with updates"	segment tree
"range update AND range query"	segment tree + lazy propagation
"count of inversions / range frequency"	segment tree (or BIT) over value or index domain

Complexity Table

Operation	Time	Space
Build	$O(n)$	$O(n)$
Point update	$O(\log n)$	—
Range query	$O(\log n)$	—
Range update (lazy)	$O(\log n)$	$O(n)$ extra for lazy array

Interview Follow-ups — Answered

- **"Segment tree vs Fenwick tree — which would you use?"** Fenwick (Section 7) is simpler and faster in practice for prefix-sum-style queries; segment tree is more general (supports min/max/gcd, arbitrary associative operations, and range updates via lazy propagation that Fenwick handles more awkwardly).
- **"Why $4n$ array size?"** A worst-case (non-power-of-2) recursion tree can need up to $4n$ nodes to stay within bounds without careful index math — it's a safe, simple upper bound.
- **"Can this support non-sum aggregates?"** Yes — any associative, combinable operation (min, max, gcd, xor) works by swapping the combine step; only the identity/no-overlap return value changes.

Common Bugs

- Forgetting to **pushDown** before recursing into children during a lazy-propagation query/update.
- Wrong "no overlap" sentinel value for non-sum aggregates (must be the identity element: $-\infty$ for max, $+\infty$ for min, 0 for xor, etc).
- Off-by-one in **mid** splitting causing infinite recursion or missed elements.

Checklist

1. Point updates only, or range updates too (need lazy propagation)?
2. What's the identity element for "no overlap" given this aggregate?
3. Is a simpler Fenwick tree sufficient (prefix sums only, no range update)?

Practice Problems

Difficulty	Problem	Source
Medium	Range Sum Query - Mutable	LeetCode 307
Medium	Range Sum Query 2D - Mutable	LeetCode 308
Hard	Count of Smaller Numbers After Self	LeetCode 315
Hard	The Skyline Problem	LeetCode 218
CSES	Range Update Queries	CSES

7 Fenwick Trees (Binary Indexed Trees)

One-line definition

A compact array-based structure that supports prefix-sum queries and point updates in $O(\log n)$ using only $O(n)$ space and far smaller constants than a segment tree.

Intuition

Index (1-based): 1 2 3 4 5 6 7 8

Each index i is responsible for a range of size ($\text{lowbit}(i) = i \& -i$)

$i=4$ (100) covers [1..4] $i=6$ (110) covers [5..6]

$i=8$ (1000) covers [1..8] $i=7$ (111) covers [7..7]

Every index "owns" a range determined by its lowest set bit; update and query both walk a chain of $O(\log n)$ such ranges using the bit trick $i \& (-i)$.

Operations

1. Point Update / Prefix-Sum Query

Complexity: $O(\log n)$ per operation, $O(n)$ space. 1-indexed array is standard.

```

vector<int> bit; int n;
void update(int i, int delta) { // add delta at index i
    for (; i <= n; i += i & (-i)) bit[i] += delta;
}
int query(int i) { // prefix sum [1..i]
    int sum = 0;
    for (; i > 0; i -= i & (-i)) sum += bit[i];
    return sum;
}
int rangeSum(int l, int r) { return query(r) - query(l - 1); }

```

2. Range Update + Range Query (difference-array trick)

Idea: maintain a BIT over the *difference array*; a range update $[l, r]$ becomes two point updates ($\text{update}(l, +\text{val})$, $\text{update}(r+1, -\text{val})$), and a prefix query on the difference BIT recovers the point value. For range-update + range-sum-query (not just point query), maintain two BITs simultaneously — a well-known but fiddly extension.

3. 2D BIT (overview)

Idea: nest the same lowbit trick in both dimensions — $\text{update}(x, y, \text{delta})$ walks $O(\log n \cdot \log m)$ cells, used for 2D range-sum queries (e.g. submatrix sum with point updates).

Patterns

Interviewer says	Think
"prefix sum with point updates"	Fenwick tree
"count inversions / smaller elements after self"	Fenwick tree over value-rank domain
"2D grid range sum with updates"	2D Fenwick tree

Complexity Table

Operation	Time	Space
Point update	$O(\log n)$	—
Prefix sum query	$O(\log n)$	—
Build from array	$O(n)$ (or $O(n \log n)$ naively)	$O(n)$
2D update/query	$O(\log n \log m)$	$O(nm)$

Interview Follow-ups — Answered

- **"Why is Fenwick faster in practice than a segment tree?"** Smaller constant factor — simple loop with bit tricks, no recursion, no $4n$ overhead, better cache behavior from the flat array.
- **"What can't a Fenwick tree do that a segment tree can?"** Fenwick fundamentally needs an *invertible* combine operation (like sum, via subtraction) to compute range queries from prefix queries — it cannot directly support min/max range queries (no inverse), and

range updates are more awkward (needs the difference-array trick or dual-BIT extension) versus a segment tree's native lazy propagation.

- **"How do you count inversions with a Fenwick tree?"** Coordinate-compress values to ranks, then sweep left-to-right, querying "how many already-inserted elements have rank $>$ current" (via prefix sum) before inserting the current rank — $O(n \log n)$ total.

Common Bugs

- Using 0-indexed arrays directly — `i & (-i)` logic requires 1-indexing (index 0 breaks the bit trick).
- Off-by-one in `rangeSum(1, r)` — forgetting the `query(1-1)` subtraction.
- Trying to use a plain Fenwick tree for min/max range queries (not supported without a different formulation).

Checklist

1. Is the operation invertible (sum/xor)? If not (min/max), use a segment tree instead.
2. 1-indexed array set up correctly?
3. Point update + range query, or range update + point query, or both (dual-BIT)?

Practice Problems

Difficulty	Problem	Source
Medium	Range Sum Query - Mutable	LeetCode 307
Hard	Count of Smaller Numbers After Self	LeetCode 315
Hard	Reverse Pairs	LeetCode 493
CSES	Dynamic Range Sum Queries	CSES
CSES	Range Update Queries	CSES

8 Advanced Tree Algorithms

One-line definition

Techniques layered on top of basic tree traversal to answer repeated structural queries (ancestor, subtree, path) in sub-linear time after preprocessing — the toolkit that separates competitive/senior-level tree questions from basic ones.

1. LCA — Binary Lifting

Idea: for each node, precompute its 1st ancestor, 2nd ancestor, 4th ancestor, 8th ancestor, and so on (powers of two). To find $\text{LCA}(u, v)$: first jump the deeper node up until both are at the same depth, then jump both nodes up together in big-to-small power-of-two hops until they're about to meet — that meeting point is the LCA. It's the same idea as binary search, just applied to "how far up do I need to jump."

Complexity: $O(n \log n)$ to precompute once, then $O(\log n)$ per query — use this when you need to answer *many* LCA queries on a tree that doesn't change (a single query is fine with the simple $O(n)$ recursive LCA from Section 1).

2. LCA — Euler Tour + Sparse Table (RMQ)

Idea: walk the whole tree with a DFS and write down every node each time you visit or leave it — this list is the "Euler tour." The LCA of two nodes turns out to be whichever node has the smallest depth between their two positions in that list. So "find the LCA" becomes "find the minimum in a range of an array" (a Range Minimum Query), which a sparse table can answer instantly.

Complexity: $O(n \log n)$ to precompute once, then $O(1)$ per query — the fastest option, but more code to set up than binary lifting.

3. Tree Diameter (general, weighted or unweighted)

Idea: BFS/DFS from any node to find the farthest node u ; BFS/DFS again from u to find the farthest node v — $\text{dist}(u, v)$ is the diameter. **Proof idea:** works because in a tree, the farthest node from *any* starting point is always one endpoint of *some* diameter. $O(n)$ time, two passes.

4. Tree DP (e.g., Maximum Independent Set / "House Robber III")

Idea: each node returns a pair (`bestIncludingMe`, `bestExcludingMe`) computed from its children's pairs — a general template for "choose a subset of nodes under an adjacency constraint" problems on trees. $O(n)$ time, $O(h)$ space.

```

pair<int,int> dp(TreeNode* r) { // {includeRoot,
    excludeRoot}
    if (!r) return {0, 0};
    auto L = dp(r->left), R = dp(r->right);
    int include = r->val + L.second + R.second;
    int exclude = max(L.first, L.second) + max(R.first, R.second);
    return {include, exclude};
}

```

5. Euler Tour Technique for Subtree Queries

Idea: DFS and record $[in[v], out[v]]$ timestamps such that v 's entire subtree occupies the contiguous range $[in[v], out[v]]$ in the tour array — turns "update/query an entire subtree" into a flat range update/query, answerable with a segment tree or Fenwick tree. $O(n)$ to build, $O(\log n)$ per subtree update/query afterward.

6. Heavy-Light Decomposition (overview)

Idea: decompose the tree into $O(\log n)$ vertical "heavy paths" such that any root-to-node path crosses at most $O(\log n)$ path segments; map each heavy path onto a contiguous array range backed by a segment tree. Enables arbitrary path queries/updates (not just root-to-node or subtree) in $O(\log^2 n)$ — the standard technique for "path sum/max between any two nodes, with updates" on large trees. Heavier to implement; typically reserved for hard/competitive-level problems.

7. Centroid Decomposition (overview)

Idea: repeatedly find the tree's centroid (a node whose removal splits the tree into pieces each $\leq n/2$ size) and recurse on each piece, building a "centroid tree" of depth $O(\log n)$. Used for divide-and-conquer over tree paths (e.g., counting paths with a given property) in $O(n \log n)$ total.

Patterns

Interviewer/problem says	Think
"many LCA queries, static tree"	binary lifting ($O(\log n)$ /query) or Euler+sparse table ($O(1)$ /query)
"diameter of a general (weighted) tree"	two-pass BFS/DFS from farthest node
"choose nodes with adjacency constraint"	tree DP, include, exclude pair per node
"update/query an entire subtree efficiently"	Euler tour flatten + segment/Fenwick tree
"path queries between arbitrary nodes, updates"	Heavy-Light Decomposition
"count/aggregate over all tree paths"	Centroid Decomposition

Complexity Table

Technique	Preprocessing	Per Query
LCA (binary lifting)	$O(n \log n)$	$O(\log n)$
LCA (Euler + sparse table)	$O(n \log n)$	$O(1)$
Tree diameter	$O(n)$ total	–
Tree DP	$O(n)$ total	–
Euler tour subtree query	$O(n) + O(\log n)/\text{op}$	$O(\log n)$
Heavy-Light Decomposition	$O(n)$	$O(\log^2 n)$ per path query
Centroid Decomposition	$O(n \log n)$	varies by problem

Interview Follow-ups — Answered

- **”Your LCA is $O(n)$ per query — can you speed it up for many queries?”** Yes — precompute binary lifting ($O(n \log n)$ once, $O(\log n)/\text{query}$) or Euler tour + sparse table ($O(1)/\text{query}$) instead of re-running the recursive LCA from scratch each time.
- **”How do you handle updates to node values with subtree-aggregate queries?”** Flatten with an Euler tour so each subtree is a contiguous range, then back it with a Fenwick/segment tree for $O(\log n)$ point update + subtree-range query.
- **”What if queries are on arbitrary paths, not just root-to-node or full subtrees?”** Heavy-Light Decomposition reduces any path to $O(\log n)$ contiguous segments, each answerable via the underlying segment tree — the general-purpose answer for ”path query + update” at scale.

Common Bugs

- Binary lifting: forgetting to first equalize depth before jumping both pointers together.
- Euler tour: recording a node only once instead of every visit/revisit (breaks the RMQ-to-LCA reduction).
- Tree DP: forgetting that ”best excluding root” still needs `max(include, exclude)` from each child (child can be included or excluded independently when root is excluded).

Checklist

1. Static tree, many queries $\Rightarrow$ preprocess (binary lifting / sparse table), don’t recompute per query.
2. Subtree query $\Rightarrow$ Euler tour flatten. Arbitrary path query $\Rightarrow$ Heavy-Light Decomposition.
3. Tree DP: does the recurrence need one state per node, or a pair/array of states (include/exclude, or per-color, etc.)?

Practice Problems

Difficulty	Problem	Source
Medium	House Robber III (tree DP)	LeetCode 337
Medium	Diameter of N-Ary Tree	LeetCode 1522
Hard	Binary Tree Cameras (tree DP)	LeetCode 968
Hard	Lowest Common Ancestor - many queries	CSES: Company Queries II
Hard	Subtree/path sum updates	CSES: Path Queries
ICPC/CP	Heavy-Light Decomposition path queries	Codeforces (HLD tag)
ICPC/CP	Centroid Decomposition path counting	Codeforces (Centroid tag)